

Security Audit

Voyager (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Voyager team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Voyager is a multi-asset trading and payment platform designed to unify digital and real-world asset markets under a single user interface. It enables users to trade cryptocurrencies, fiat currencies, and tokenized commodities such as gold, silver, and platinum. Voyager incorporates institutional-grade security infrastructure and aims to deliver seamless functionality for trading, spending, and borrowing.

Project Name: Voyager

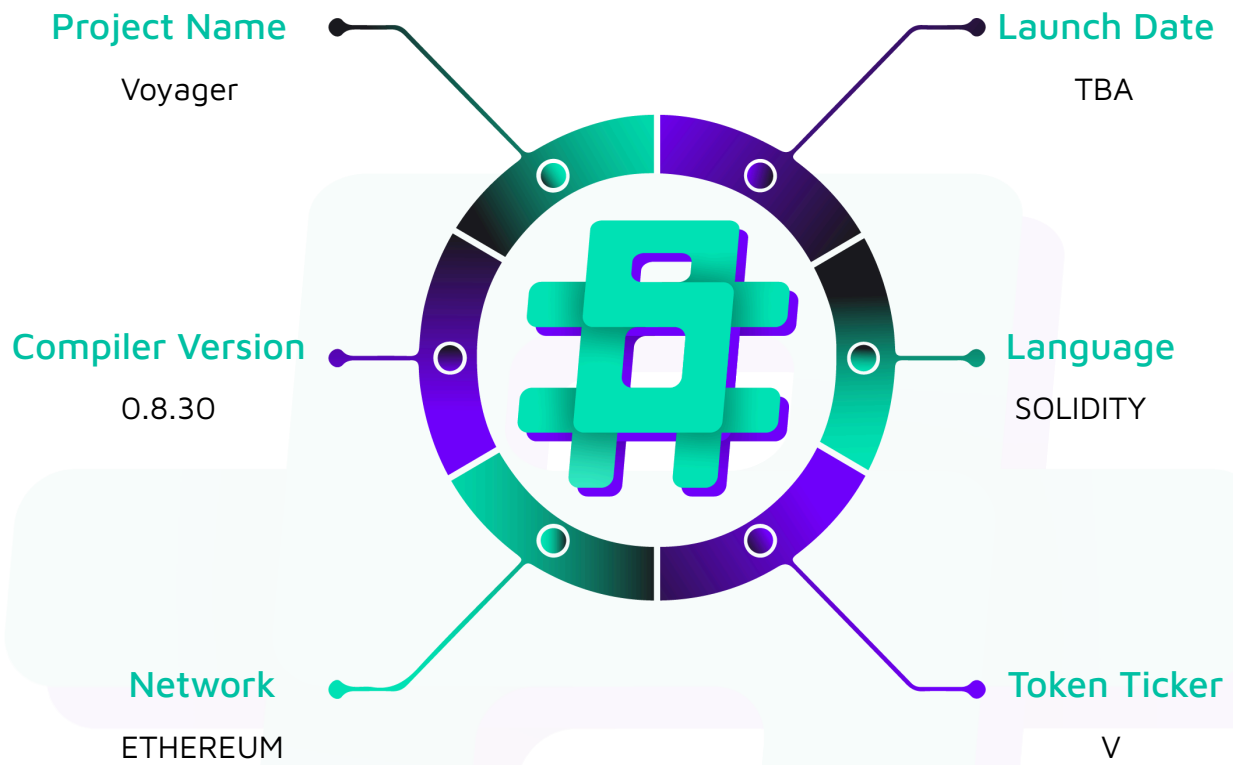
Project Type: Token

Compiler Version: 0.8.30

Website: <https://voyager.cx/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the Voyager project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Voyager Smart Contracts
Platform	Ethereum / Solidity
Audit Date	August, 2025
Contract 1	Voyager.sol
Contract 1 MD5 Hash	d215f44138a69b185b020a4044f194c0
Audited GitHub Commit Hash	734bf1a8f55284fdc6f001b56728a5b9ece77709
Fix Review GitHub Commit Hash	5d5fbae4fdb0d9cdffa82be5ba5e2719f989ddf9
Contract Address	0xe599c80DB30d674d27Eaab917C0f5699a8850B12

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 Low severity vulnerabilities

3 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Voyager.sol Allows users to: - Transfer token. - Approve the token. - Use permit for approvals.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Voyager project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Voyager project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] Voyager#permit - Signature malleability due to missing ECDSA checks

Description

The `permit` function in Voyager uses ``ecrecover`` to validate signatures without implementing checks for signature malleability. It has no validation that ``s`` is in the lower half of the curve's order. While the nonce mechanism (`_nonces[owner]++`) prevents signature replay attacks, the lack of proper ECDSA checks remains a deviation from best practices.

Recommendation

It's recommended to require the `s` value to be in the lower half order. Reference: [link](#)

Status

Resolved

[L-02] Voyager - Centralization Risks

Description

The current implementation allows the Owner to have excessive control, including the ability to withdraw tokens or ETH from within the contract without any restrictions through `withdrawStuck()` and the ability to disable trading without being tied to any timelock. This concentration of control introduces centralization risks, as a single compromised or malicious administrator could disrupt or manipulate the contract, leading to potential security breaches or operational failures.

Recommendation

If it's not already planned, consider establishing a multi-signature wallet to control the owner role and/or integrating time locks for critical administrative functions like enabling/disabling trading and changing the owner.

Note

The Voyager team informed Hashlock that they have implemented a Gnosis Safe multisig for the owner role, mitigating the risk. As this is a pre-production contract, the deployed contracts will not present this vulnerability.

Status

Resolved

[L-03] Ownable#transferOwnership - Consider Two-step Ownership Transfer

Description

In Ownable contract, ownership is transferred within one function call, namely `transferOwnership`. This pattern comes with its risks; any mistake can lead to losing the ability to enable/disable trading and withdraw stuck funds in the Voyager contract.

```
function transferOwnership(address newOwner) public virtual onlyOwner {  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    emit OwnershipTransferred(_owner, newOwner);  
    _owner = newOwner;  
}
```

Recommendation

Consider implementing a two-step ownership transfer for a safer ownership transfer. This requires two functions:

1. Assigning the next owner: The Current owner proposes a `new_owner`.
2. Accepting ownership: `new_owner` accepts ownership.

Status

Resolved

QA

[Q-01] Voyager - Inconsistent Error Handling Patterns

Description

The contract exhibits inconsistent error-handling patterns throughout its codebase. While the main contract functionality extensively uses custom errors (defined in lines 94-112) with revert statements for gas-efficient and clear error handling, the inherited Ownable contract uses traditional require statements. See:

```
modifier onlyOwner() {  
    require(_owner == _msgSender(), "Ownable: caller is not the owner");  
    _;  
}  
  
...  
  
function transferOwnership(address newOwner) public virtual onlyOwner {  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    emit OwnershipTransferred(_owner, newOwner);  
    _owner = newOwner;  
}
```

This inconsistency creates a mixed error handling approach.

Recommendation

Standardize the error handling approach throughout the contract by converting the require statements in the Ownable contract to use custom errors.

Status

Resolved

[Q-02] Voyager#transfer, transferFrom - Confusing Parameter Naming

Description

Using `sender` and `recipient` parameters in `transfer` and `transferFrom` functions instead of the more conventional `from` and `to` naming creates inconsistency and confusion. The `withdrawStuck()` function already uses the conventional `to` parameter for the receiving address, making the naming inconsistent across the contract. Using `from` instead of `sender` would reduce confusion in lines such as:

```
uint256 currentAllowance = _allowances[sender][_msgSender()];
```

Recommendation

Standardize parameter naming by using `from` - `to` instead of `sender` - `recipient` throughout the contract to align with ERC20 conventions and improve code readability.

Status

Resolved

[Q-03] Voyager#withdrawStuck - Improper handling of ERC20 `transfer` return value

Description

The `Voyager::withdrawStuck()` function attempts to transfer stuck ERC20 tokens without properly handling the return value of the transfer call.

Vulnerability Details

In the `Voyager::withdrawStuck()` function, the contract transfers stuck ERC20 tokens to address `to` as follows:

```
} else {  
  
    // Withdraw ERC20 tokens  
  
    IERC20(token).transfer(to, amount);  
  
    emit StuckTokensWithdrawn(token, to, amount);  
  
}
```

This function call has two major issues:

1. Non-compliant ERC20 tokens can revert

The contract assumes that all ERC20 tokens implement the standard transfer function with the following signature:

```
function transfer(address recipient, uint256 amount) external returns (bool);
```

2. Tokens that return false on failure are ignored

Certain ERC20 tokens (e.g., USDT) do not revert on failure but return false instead. Since the return value of transfer is not checked, if a transfer fails, the function continues execution, assuming the transfer was successful.

Impact

Failure to properly handle the ERC20 transfer return value can lead to unexpected reverts or unnoticed transfer failures, resulting in potential denial-of-service conditions or stuck tokens, as detailed in the vulnerability details section.

Recommendation

It is recommended to use OpenZeppelin's SafeERC20 for transfers or manually implement similar functionality to `safeTransfer`.

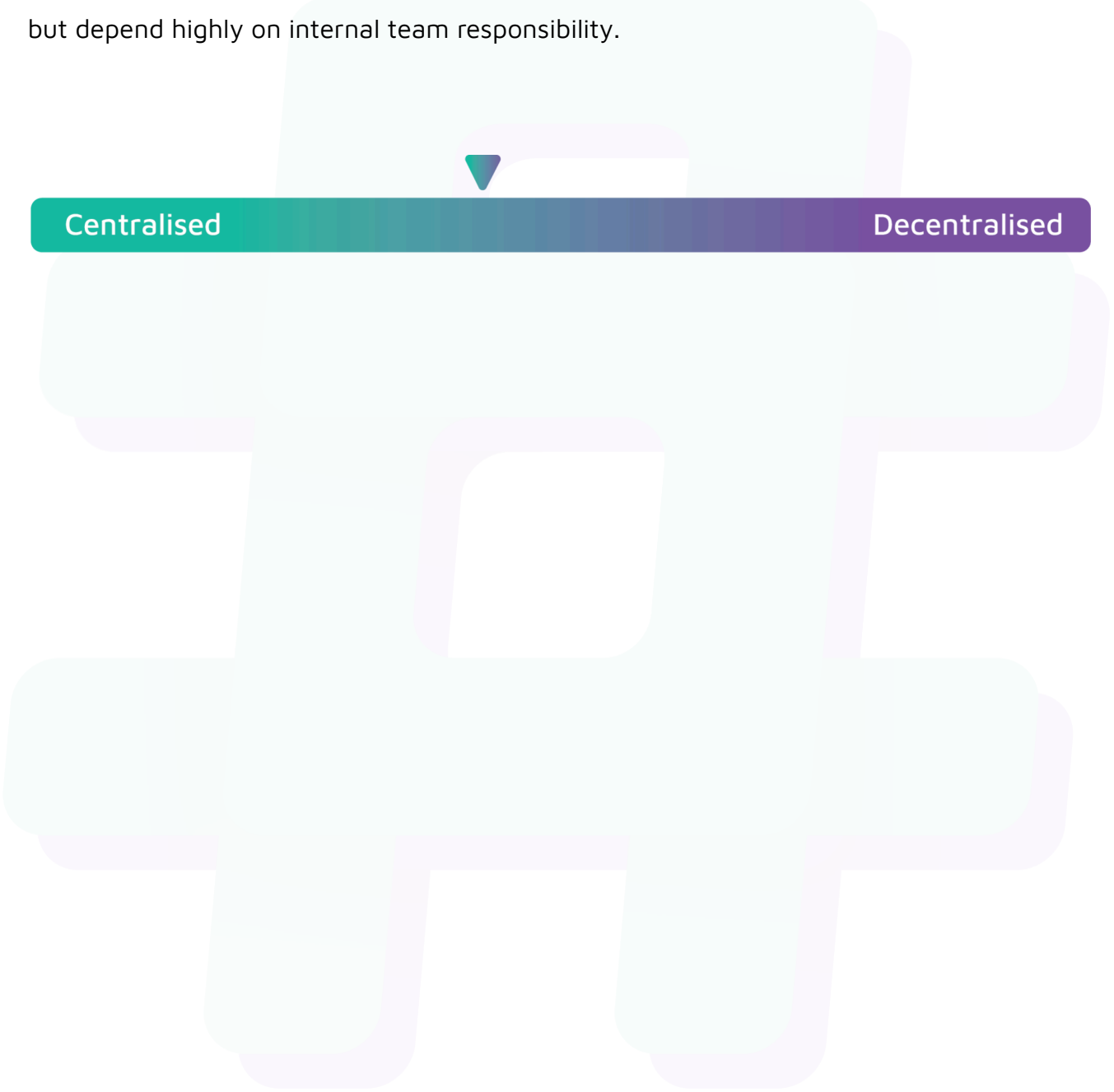
Status

Resolved

Centralisation

The Voyager project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Voyager project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.